강화학습 Day04 hw1 보고서

작성자: Doyeon Park

작성일: 2026.11.2

목차

1. 과제 개요
2. 학습 방법
3. 실험 설계
4. 실험 결과
5. 결론

1. 과제 개요

1.1 과제 목표

본 프로젝트는 MuJoCo 기반 Humanoid-v5 환경에서 PPO 의 학습 과정을 분석하고, 학습률을 변경했을 때 성능과 행동이 어떻게 달라지는지 확인하는 것을 목적으로 하였다. Stable-Baselines3 의 공개 구현을 활용했으며, 알고리즘 원본은 수정하지 않았다. 환경 생성, 학습, 모델 저장과 재로딩, 별도 평가, 그래프 및 영상 생성 코드를 구성해 실행했다.

기본 조건 A 의 학습률은 0.0003, 변경 조건 B 의 학습률은 0.0001 로 설정했다. 두 조건을 각각 1,001,472 환경 step 을 학습하고, 동일한 초기 상태 seed 목록으로 모델별 20 회씩 평가했다.

평가 결과 평균 보상은 A 가 496.01, B 가 479.96 이었고, 평균 유지 시간은 A 가 1.463 초, B 가 1.410 초였다. 이번 학습 seed 와 예산에서는 낮은 학습률에 따른 개선 효과를 확인하지 못했으며, 안정적인 보행도 확보하지 못했다.

1.2 실험 환경

환경의 목표는 몸통의 정상 높이를 유지하면서 +x 방향으로 전진하는 것이다. 물리 계산 간격은 0.003 초이며, 환경 한 step 마다 물리 계산을 5 회 수행한다. 따라서 환경 한 step 은 시뮬레이션 시간 0.015 초에 해당한다. 화면을 띄우지 않아도 MuJoCo 가 중력, 관절 움직임, 접촉을 계산한다.

기본 관측은 총 348 차원으로 구성된다.

1, qpos[2:]의 22 개 값은 몸통 높이와 방향, 관절 자세를 나타낸다.
2, qvel 의 23 개 값은 몸체와 관절의 속도를 나타낸다.
3, cinert 의 130 개 값은 몸체의 관성과 질량 등의 정보다.
4, cvel 의 78 개 값은 몸체별 공간 속도를 나타낸다. 다섯째, qfrc_actuator 의 17 개 값은 구동기가 관절에 만드는 일반화 힘이다.
5, cfrc_ext 의 78 개 값은 몸체별 외력과 모멘트다.

기본 관측에서는 몸통의 절대 x, y 위치를 제외한다. 관절 속도는 같은 자세에서도 움직임의 방향을 구별하는 데 사용될 수 있으며, 외력 정보는 접촉 상태를 파악하는 데 사용될 수 있다. 몸통 방향을 나타내는 4 개 성분은 quaternion 이며, 각각을 독립적인 관절 각도로 해석하면 안 된다.

1.3 관측과 행동

행동 공간은 Box(-0.4, 0.4, (17,), float32)이며, 17 개의 연속 제어 입력을 동시에 결정한다.
제어 대상은 복부 3 축, 양쪽 엉덩관절 각각 3 축, 양쪽 무릎 각각 1 개, 양쪽 어깨 각각 2 축, 양쪽 팔꿈치 각각 1 개다.

행동은 목표 관절 각도를 지정하는 값이 아니라 motor 제어 입력이다. 모델 XML 에서 gear 는 팔 25, 복부와 일부 엉덩관절 100, 무릎 200, hip_y 300 으로 다르다. 따라서 같은 크기의 입력이라도 관절에 미치는 효과가 다를 수 있다.

1.4 보상과 종료 조건

정상 상태 여부를 H 라고 할 때, 몸통 높이 z 가 1.0m 보다 크고 2.0m 보다 작으면 H 는 1 이며, 그렇지 않으면 0 이다.

전진 속도 vx 는 한 step 동안 로봇 전체 질량중심의 x 좌표 변화량을 시뮬레이션 시간 0.015 초로 나눈 값이다.
보상은 다음과 같이 계산된다.

(보상) = 5H + 1.25 × (전진 속도) − (제어 비용) − (접촉 비용)
(제어 비용) = 0.1 × (실제 제어 입력의 제곱합)
(접촉 비용) = min(5 × 10^(-7) × 외력·모멘트 제곱합, 10)

자세 유지 보상은 정상 높이면 5, 아니면 0 이다.

전진 보상은 몸통 좌표 한 점이 아니라 전체 모델의 질량중심 이동을 기준으로 계산한다. 따라서 info 에 기록되는 몸통의 x_position 변화량과 보상 계산에 사용하는 질량중심 이동량을 구분해야 한다.

제어 비용은 실제 입력 크기의 제곱에 비례하므로, 입력 부호보다 크기에 영향을 받는다. 접촉 비용은 외력과 모멘트의 제곱합에 비례한다.

실제 구현에서 control_cost 는 함수 인자 action 대신 data.ctrl 을 사용한다. 별도의 무작위 행동 100 step 에서 위 수식과 실제 reward 를 비교한 결과, 최대 오차는 약 8.9 × 10^(-16)으로 부동소수점 반올림 수준이었다.

기본 설정은 terminate_when_unhealthy=True 다. 따라서 몸통 높이가 정상 범위에서 벗어나면 terminated=True 가 된다.

시간 제한은 별도의 TimeLimit 래퍼가 담당한다. 기본 제한은 1,000 step 으로, 시뮬레이션 시간 15 초에 해당한다. 제한에 도달하면 truncated=True 가 된다.

건강 상태에 따른 종료와 시간 제한에 따른 중단은 다른 신호이며, 같은 step 에서 둘 다 참일 수 있다. 검증용으로 시간 제한을 2 step 으로 설정한 별도 환경에서는 terminated=False, truncated=True 를 확인했다. 본 실험의 시간 제한은 변경하지 않았다.

초기화 함수 reset_model 은 초기 위치·자세 qpos 와 속도 qvel 에 각각 [-0.01, 0.01] 범위의 균등 잡음을 더한다. 같은 reset seed 에서 qpos 와 qvel 이 일치함을 확인했다.

학습 seed 는 두 조건 모두 0 으로 설정했고, 최종 평가 seed 는 2000 부터 2019 까지 사용했다. 무작위 정책 확인에서는 행동 공간의 난수도 별도로 고정했다.

높은 총보상은 정상 높이를 오래 유지한 결과일 수 있기 때문에 총 보상만으로는 자연스러운 보행을 했는지에 대한 여부를 알 수는 없다. 따라서 자연스러운 보행 여부는 총보상뿐 아니라 전진 속도, 유지 시간, 제어 입력, 영상을 함께 확인해야 한다.

2. 학습 방법

2.1 DQN 과 PPO 비교

Day03 의 CartPole DQN 은 왼쪽과 오른쪽이라는 두 행동 중 하나를 선택했다. 반면 Humanoid PPO 는 17 개의 연속 제어 입력을 동시에 결정한다.

DQN 은 행동 가치 Q(s, a)를 학습하며, 일반적으로 행동별 Q 값을 출력해 가장 큰 값을 선택한다. PPO 는 행동 분포를 표현하는 정책과 상태 가치 V(s)를 함께 학습한다.

Humanoid 의 각 입력을 단 3 단계로만 이산화해도 가능한 조합은 3^17, 즉 129,140,163 개다. 이산화한 DQN 을 사용하는 것이 원리적으로 불가능하다는 뜻은 아니지만, 단순 확장은 행동 조합 증가와 제어 정밀도 손실을 일으킨다. 이번에는 연속 행동 분포를 사용하는 PPO 를 적용했다.

경험 사용 방식에도 차이가 있다. DQN 은 Replay Buffer 에 보관한 과거 경험을 재사용한다. PPO 는 최근 정책으로 수집한 Rollout Buffer 를 사용하고, 해당 경험을 여러 epoch 동안 학습한 뒤 새로운 경험을 수집한다.

2.2 신경망 구조

실제 모델의 Actor 는 관측 348 개를 입력받아 64 개 노드의 은닉층 두 개를 거친 뒤, 행동 평균 17 개를 출력한다.

Critic 도 관측 348 개를 입력받아 64 개 노드의 은닉층 두 개를 거친 뒤, 상태 가치 1 개를 출력한다. 참고로 두 MLP 의 은닉층 활성화 함수는 Tanh 다.

Actor 의 표준편차는 상태와 무관한 학습 파라미터 log_std 17 개로 표현되며, std=exp(log_std)로 계산한다. 따라서 매 상태에서 신경망이 평균과 표준편차를 모두 별도로 출력하는 구조는 아니다.

저장한 모델을 불러와 배치 1 개를 입력했을 때 행동의 shape 는 (1, 17), 가치의 shape 는 (1, 1), log_prob 의 shape 는 (1,)이었다.

Critic 은 즉시 보상이나 성공 확률을 출력하는 것이 아니라, 현재 정책을 따랐을 때 얻을 미래 할인 보상을 추정한다. 학습에서는 Gaussian 분포에서 행동을 샘플링하며, 최종 평가에서는 평균 행동을 사용한다.

2.3 경험 수집

학습 코드의 model.learn 은 OnPolicyAlgorithm.learn 으로 연결된다. 이 함수는 collect_rollouts 로 경험을 수집한 뒤 PPO.train 으로 신경망 가중치를 갱신하는 과정을 반복한다.

수집 중 policy(obs)는 행동, 가치, log_prob 를 출력한다. 환경에는 허용 범위로 잘라낸 행동을 전달한다. 반면 버퍼에는 샘플링한 원래 행동과 당시 log_prob 를 보관하며, 이후 확률 비율 계산에 사용한다.

핵심 코드에서 다음 호출이 이 연결을 보여준다.

```
actions, values, log_probs = self.policy(obs_tensor)
new_obs, rewards, dones, infos = env.step(clipped_actions)
```

관측 obs_tensor 가 행동·가치·로그밀도로 변환되고, 제한된 행동이 다음 관측과 보상으로 이어진다.

버퍼에는 관측, 행동, 보상, 에피소드 시작 표시, 가치, log_prob 를 저장한다. 환경 1 개에서 observations 의 크기는 (2048, 1, 348), actions 는 (2048, 1, 17), 보상과 가치 배열은 (2048, 1)이다.

하나의 rollout 은 하나의 에피소드와 같지 않으며, 여러 에피소드를 포함할 수 있다. 다음 경험 수집을 시작할 때 버퍼를 초기화하지만, 현재 rollout 은 10 epoch 동안 재사용한다.

2.4 학습 과정

Return 은 Critic 을 학습시키기 위한 미래 보상 목표다. Advantage 는 수집한 행동 결과가 해당 상태의 기준 가치보다 얼마나 유리했는지를 나타내는 추정값이다.

이번 SB3 구현은 GAE 를 사용하므로, returns 가 단순한 에피소드 보상 총합과 항상 같지는 않다.

계산식은 다음과 같다.
$\delta_t = r_t + \gamma \times m_t \times V(s_{t+1}) - V(s_t)$
$A_t = \delta_t + \gamma \times \lambda \times m_t \times A_{t+1}$
$Return_t = A_t + V(s_t)$

이번 설정은 γ=0.99, λ=0.95 다. m_t 는 실제 에피소드 경계를 넘어 다음 에피소드의 값을 섞지 않도록 하는 마스크다.

RolloutBuffer.compute_returns_and_advantage 는 마지막 step 부터 역순으로 값을 계산한다. 경험 수집 구간만 끝났고 에피소드가 계속되는 경우에는 마지막 상태 가치를 이용해 이후의 보상을 추정한다.

순수 시간 제한 중단은 collect_rollouts 에서 마지막 관측 가치를 보정하여 실제 종료와 구분한다. 미니배치 학습에서는 Advantage 를 평균과 표준편차로 정규화한다.

실제 코드의 다음 문장이 Return 과 Advantage 의 관계를 보여준다.

self.returns = self.advantages + self.values

PPO 는 새 정책과 이전 정책 사이의 행동 확률밀도 비율을 사용한다.

ratio = exp(new_log_prob − old_log_prob)

정책 손실은 ratio 와 Advantage 를 곱한 항, 그리고 ratio 를 제한한 뒤 Advantage 를 곱한 항 중 작은 값을 선택하여 계산한다.

$L_{policy} = -\text{mean}[\min(\text{ratio} \times A, \text{clip}(\text{ratio}, 0.8, 1.2) \times A)]$

ratio 는 같은 상태와 수집 행동에 대해 새 정책의 확률밀도를 이전 정책의 확률밀도로 나눈 값이다. 연속 행동이므로 특정 값의 점확률이 아니라 확률밀도를 사용한다.

이번 clip_range 는 0.2 다. 이는 이미 유리한 방향으로 크게 바뀐 표본에서 추가적인 학습 이득을 제한하는 역할을 한다. 모든 실제 ratio 나 가중치를 반드시 20% 범위 안에 가두는 강제 제약은 아니며, 성능 향상을 보장하지도 않는다.

실제 PPO.train 에서는 다음과 같이 ratio 를 계산한다.

ratio = th.exp(log_prob − rollout_data.old_log_prob)

이번 코드에는 서로 다른 세 종류의 제한이 있다.

행동 clipping 은 환경에 보내는 입력을 [-0.4, 0.4]로 제한한다. PPO ratio clipping 은 정책 목표에서 확률밀도 비율을 제한한 항을 사용한다. Gradient norm clipping 은 역전파로 계산한 기울기의 norm 을 최대 0.5 로 제한한다. 세 기능은 서로 구분해야 한다.

이번 설정에서 총손실은 다음과 같다.

총손실 = 정책 손실 + 0.5 × 가치 MSE + 0 × 엔트로피 손실

정책 손실은 Advantage 가 유리한 행동 쪽으로 정책을 수정한다. 가치 손실은 Critic 의 예측을 Return 목표에 맞춘다.

엔트로피 계수 ent_coef 는 0 이므로 명시적인 엔트로피 보너스는 가중치 갱신에 기여하지 않는다. 다만 Gaussian 에서 행동을 샘플링하므로 탐험 자체는 존재한다. 가치 예측에 대한 별도의 PPO clipping 은 설정하지 않았다.

PPO.train 은 수집한 2,048 개의 경험을 섞어 64 개씩 나누고, 10 epoch 동안 Adam 으로 가중치를 갱신한다. 한 rollout 에서 미니배치는 32 개이며, optimizer step 은 32 × 10 으로 총 320 회 수행된다.

전체 학습은 489 rollout 이므로 optimizer step 은 156,480 회다. 저장한 모델의 optimizer 카운터에서도 이 값을 확인했다.

3. 실험 설계

3.1 비교조건

실험 질문은 “PPO 학습률을 0.0003 에서 0.0001 로 낮추면 같은 환경 step 에서 성능과 행동이 어떻게 달라지는가?” 이다.

작은 학습률은 한 번의 업데이트 폭을 줄여 정책 변화가 완만해질 수 있지만, 제한된 학습 예산에서는 학습 진행이 느려질 수 있다는 가설을 세웠다.

기본 조건 A 의 학습률은 0.0003, 변경 조건 B 의 학습률은 0.0001 로 설정했다. 학습률 외의 주요 조건은 동일하게 유지했다.

두 조건 모두 처음부터 학습했으며, 시험 학습 checkpoint 를 이어 사용하지 않았다. 각 조건의 학습 예산은 1,001,472 환경 step 이다. PPO rollout 크기인 2,048 의 정수배로 설정하여 실제 수집량이 두 조건에서 같도록 했다.

학습 seed 는 0 이며, Humanoid-v5 의 기본 보상과 환경 1 개를 사용했다. 정책은 MlpPolicy, 계산 장치는 CPU, PyTorch 스레드는 1 개로 설정했다. 관측과 보상 정규화는 사용하지 않았다.

PPO 의 주요 설정은 n_steps=2048, batch_size=64, n_epochs=10, gamma=0.99, gae_lambda=0.95 다. 나머지 주요 설정은 clip_range=0.2, vf_coef=0.5, ent_coef=0, max_grad_norm=0.5 다.

최종 저장 파일을 다시 불러와 별도의 환경에서 모델별 20 회 평가했다. 두 모델에 동일한 평가 seed 2000~2019 를 사용했고, deterministic=True 로 실행했다. 평가 중에는 학습하지 않았다.
또한 동일한 관측에서 저장 전후의 결정적 행동이 일치함을 확인했다.

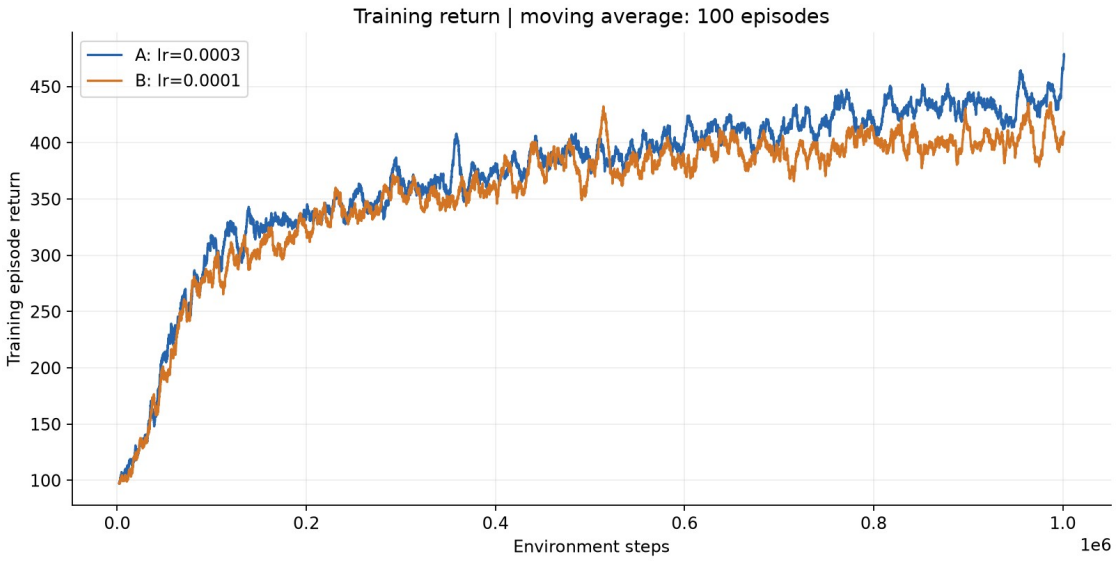
3.2 실행 환경과 설정

사용한 CPU 는 Intel Core Ultra 7 155H 다. 그래픽카드가 장착되어 있으나, 이번 학습은 CPU 로 실행했다.

OS 기록은 Linux 7.0.0-38-generic x86_64 이며 Python 버전은 3.12.3 이다. 주요 패키지 버전은 Gymnasium 1.4.0, MuJoCo 3.17.0, Stable-Baselines3 2.9.2, PyTorch 2.15.0+cpu, NumPy 2.5.4 다.

학습 시간은 A 가 1,426.65 초, B 가 1,366.29 초였다. 시간은 코드에서 모델 생성을 시작한 시점부터 learn 이 종료된 시점까지 측정했다.

4. 실험 결과



위 그림은 환경 step 에 따른 학습 에피소드 보상을 나타낸다. 최근 100 개 완료 에피소드의 이동평균을 사용했다.

두 조건 모두 초기 약 100 점 수준에서 보상이 상승했다. 후반에는 A 의 이동평균이 B 보다 높은 구간이 많았다.

다만 이 곡선은 학습 중 확률적 행동으로 수집한 에피소드 보상이다. 에피소드 길이가 다르므로, 100 개 에피소드가 포함하는 환경 step 의 폭도 다를 수 있다.

모델별 최종 20 회 평가 결과는 다음과 같다.

총보상은 A 가 평균 496.01, 표준편차 93.72 였고, B 가 평균 479.96, 표준편차 86.69 였다.
유지 step 수는 A 가 평균 97.55, 표준편차 18.56 이었고, B 가 평균 94.00, 표준편차 18.29 였다.
유지 시간은 A 가 평균 1.463 초, 표준편차 0.278 초였고, B 가 평균 1.410 초, 표준편차 0.274 초였다.
전진 속도는 A 가 평균 0.4103m/s, 표준편차 0.1309m/s 였고, B 가 평균 0.4156m/s, 표준편차 0.1243m/s 였다.

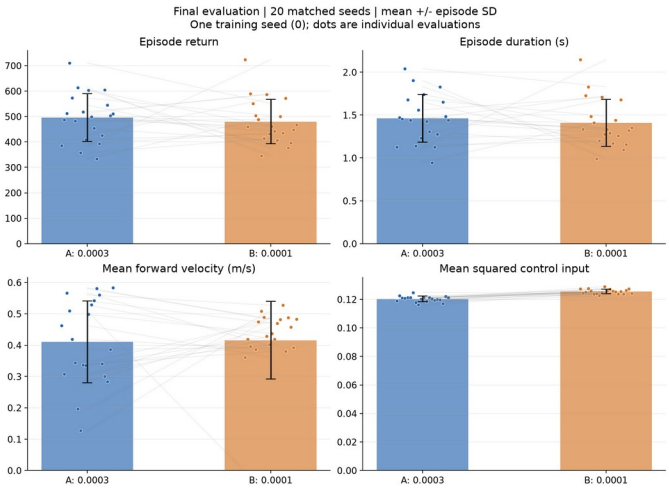
제어 입력 제곱 평균은 A 가 0.12030, 표준편차 0.00191 이었고, B 가 0.12545, 표준편차 0.00161 이었다.
두 조건 모두 20 회 평가에서 terminated 가 발생했고, truncated 는 발생하지 않았다.

여기서 표준편차는 평가 에피소드 간 모집단 표준편차로, ddof=0 을 사용했다. 재학습 실행 사이의 편차를 나타내는 값은 아니다.

속도는 매 step 의 질량중심 전진 변위를 합한 뒤 시뮬레이션 유지 시간으로 나누어 계산했다. 제어 입력 지표는 실제 전달된 입력의 제곱합을 step 수와 행동 차원 17 의 곱으로 나누었다.

각 에피소드의 지표를 계산한다

4.1 결과비교



위 그림은 최종 평가의 평균, 표준편차, 개별 결과를 보여준다. 회색 선은 같은 평가 seed 에서 얻은 A 와 B 의 결과를 연결한다.

B 는 A 보다 평균 보상이 16.05 점 낮았고, 평균 유지 시간이 0.053 초 짧았다. 평균 전진 속도는 0.0053m/s 높았으며, 제어 입력 제곱 평균은 약 0.00516 높았다.

두 모델 모두 평가 20 회에서 건강 상태에 따른 종료가 발생했다. 시간 제한까지 자세를 유지한 사례는 없었다.

이번 결과에서는 낮은 학습률이 같은 예산에서 더 좋은 보행을 만든다는 근거를 얻지 못했다. 두 조건의 차이는 평가 변동에 비해 작았다.

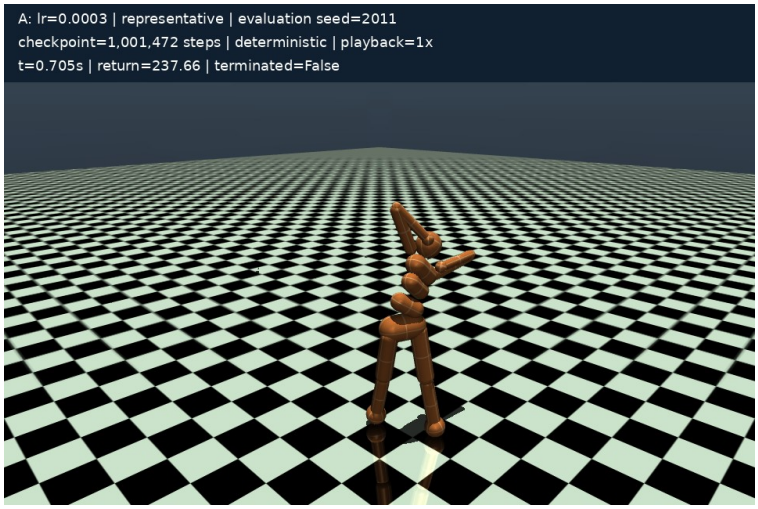
B 의 평가 표준편차가 일부 지표에서 낮았지만, 이를 재학습 안정성 향상으로 해석하기는 어렵다.

실패한 원인으로는 제한된 수집 예산에서의 학습 진행 차이, 확률적 탐험과 최종 평균 행동의 차이 등을 생각해볼 수 있다. 하지만 이 모든것은 추측이다.

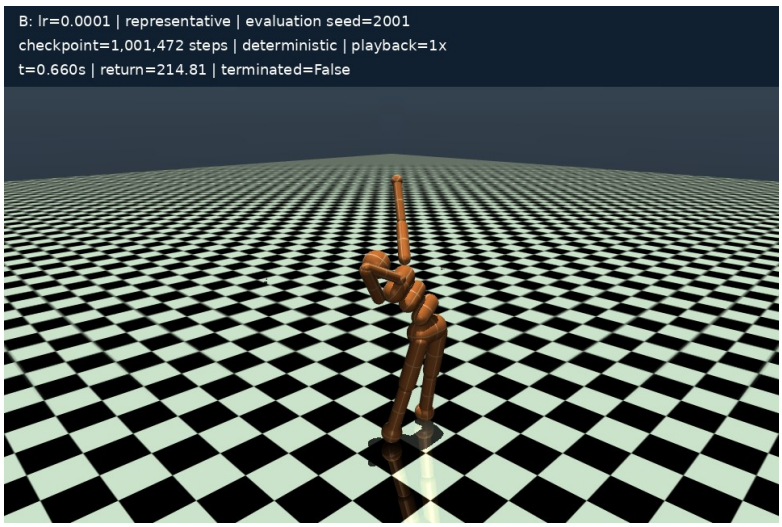
4.2 보행 영상 분석

영상은 각 조건의 평가 20 회 중 보상 중앙값에 가장 가까운 에피소드 1 개와 최저 보상 에피소드 1 개를 선정했다. 동률이면 작은 seed 를 선택했다.

중앙값 대표 사례는 성공 사례를 의미하지 않는다. 또한 조건별로 선택된 seed 가 다르므로, 대표 영상끼리 같은 초기 조건에서의 직접 비교로 해석하지 않는다.



A 의 대표 사례는 평가 seed 2011 이다. 총 95 step, 시뮬레이션 시간 1.425 초 동안 유지했으며 보상은 498.96 이었다.



B 의 대표 사례는 평가 seed 2001 이다. 총 89 step, 시뮬레이션 시간 1.335 초 동안 유지했으며 보상은 459.62 였다.

A 의 최저 보상 사례는 seed 2013 으로, 63 step 동안 실행되었고 보상은 333.59 였다. B 의 최저 보상 사례는 seed 2006 으로, 66 step 동안 실행되었고 보상은 345.92 였다.

영상으로 재실행한 4 개 사례 모두 보상과 step 수가 원래 평가 기록과 일치했다. 이 사례들의 최종 몸통 높이는 약 0.974~0.999m 로, 정상 높이의 하한인 1m 를 벗어났다.

영상은 최종 checkpoint 인 1,001,472 step 모델을 사용했다. 조건, 평가 seed, checkpoint, 1 배속 여부를 영상 상단에 표시했으며, 66.6667fps 로 시뮬레이션 시간과 재생 시간을 맞췄다.

환경은 몸통 높이가 경계를 벗어나면 바로 종료되므로, 완전히 바닥에 쓰러지는 장면이 없을 수 있다.

5. 결론

5.1 결과 요약

Humanoid-v5 의 관측 348 차원, 행동 17 차원, 보상, 종료, 초기화 조건을 설치 코드와 실행 결과로 확인했다.

PPO 가 경험을 수집하고 GAE 목표를 계산한 뒤, clipping 을 포함한 손실로 정책과 Critic 을 갱신하는 경로를 추적했다. 저장한 모델을 이용해 신경망 구조, 버퍼 shape, optimizer 갱신 횟수도 확인했다.

학습률만 변경한 두 조건을 동일한 환경 step 예산으로 비교했다. 학습 중 보상은 상승했지만, 최종 평균 유지 시간은 약 1.4 초에 그쳤다. 이번 학습 seed 와 예산에서는 낮은 학습률의 개선 효과와 안정적인 반복 보행을 확인하지 못했다.

5.2 실험의 한계

첫째, 학습 seed 가 하나다. 평가 20 회는 초기 상태 변화에 따른 해당 모델의 변동을 보여주지만, 신경망 초기화와 학습 난수에 따른 재학습 편차는 추정하지 못한다.

둘째, 약 100 만 step 의 제한된 예산에서 관측한 결과다. 이를 장기 성능이나 수렴 성능으로 일반화하지 않는다.

셋째, 영상은 중앙값 대표 사례와 최저 보상 사례만 포함한다. 전체 20 회 행동의 모든 차이를 대표하지는 못한다.

넷째, 제어 입력 제곱 평균은 실제 일률이나 에너지 소모량이 아니다. 따라서 이번 지표만으로 에너지 효율을 판단할 수 없다.

다섯째, 최종 모델 평가를 중심으로 분석했으며, 중간 checkpoint 별로 같은 조건의 평가 곡선을 별도로 확보하지 않았다.

5.3 추가 실험

우선 A 와 B 의 설정을 고정한 채 추가 학습 seed 를 양쪽에 동일하게 적용하여 재학습 결과를 비교할 수 있다. 예를 들어 seed 1 을 추가하면 기존 seed 0 결과와 함께 학습 seed 에 따른 편차를 확인할 수 있다.

다음으로 별도 실험에서 두 조건의 환경 step 예산을 같은 비율로 늘려 학습 예산의 영향을 확인할 수 있다.

학습률, 보상, 신경망 크기 등을 동시에 변경하면 원인 해석이 어려워진다. 따라서 질문별로 변경 변수와 고정 변수를 구분하여 실험해야 한다.

5.4 공개 코드와 AI 활용 범위

환경과 PPO 구현은 Gymnasium 과 Stable-Baselines3 를 활용했다.
연결 스크립트, 설정 파일, 평가·그래프·영상 코드, 코드 분석 정리에 Codex 의 도움을 받았다.
보고서의 수치는 저장된 실제 실행 기록에 근거한다.

참고문헌

[1] Farama Foundation. Gymnasium 1.4.0. Humanoid-v5 구현 및 humanoid.xml.
Gymnasium 저장소
Humanoid 환경 문서
[2] Stable-Baselines3 2.9.2. PPO 및 공통 정책·버퍼 구현.
Stable-Baselines3 저장소
PPO 문서
[3] Schulman et al. Proximal Policy Optimization Algorithms. 2017.
논문
[4] Schulman et al. High-Dimensional Continuous Control Using Generalized Advantage Estimation.
논문